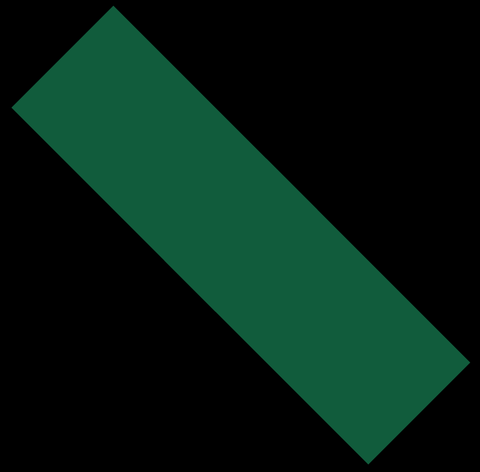
■ MDEVCAMP — 2026.06.04 — 15:35 — LOOP

TAMING THE WEB WITH KMP.



JACKSON MAFRA — MOBILE / UMAIN // A KMP LIBRARY SHIPPED TO 4 PLATFORMS — INCLUDING THE WEB

▶ // SYSTEM USER PROFILE — LOADING

JACKSON MAFRA.

MOBILE ENGINEER / UMAIN

Digital product studio · Stockholm · 2001 · part of Eidra

// ANDROID PRIORITY · MOBILE SECURITY

// BREAKING THINGS · PEN TESTING · WRITING · OBSESSED W/ FAILURE MODES

```
// /about.log
$  cat ~/about.log
"Mobile engineer at Umain.
 Mostly Android.
 Reading. Writing.
 Pen testing.
 Mobile security path."

$  ls ~/profile/links
github.com/jacksonmafra
linkedin.com/in/jacksonmafra
medium.com/@jacksonmafra
```

SCAN — JUMP TO ABOUT.ME

https://about.me/jacksonfdam

// SL — STOCKHOLMS LOKALTRAFIK

REAL BACKEND.
REAL CASE STUDY.

// WHAT IS SL

Stockholm's public-transport authority.

Trains, metro, trams, buses, ferries — and one of 20+ Swedish regional operators.

Data hub: trafiklab.se (Samtrafiken · Trafikverket)

// WHY THIS API

Not a hello-world. Real archetypes:

- 5 REST features + realtime trip stream
- Lines · Sites · Departures · StopPoints
- Same shapes you ship in your product

// ONE LIBRARY **com.umain.transport:stockholm-transport → Android · iOS · JVM · Node · Browser**

// THE EXAMPLES YOU'LL SEE TODAY ALL HIT THIS LIBRARY

// QUICK BRIEFING — KOTLIN/JS

ONE COMPILER. FIVE TARGETS.

01	KOTLIN BACKEND SHARE	DTOs · validation · REST abstractions	
02	MOBILE + WEB CLIENT SHARE	Same domain layer in Android · iOS · JS	← THIS LIBRARY
03	FRONTEND UI IN KOTLIN	kotlin-wrappers (React) · Compose Web · Kobweb	
04	OLDER-BROWSER FALLBACK	Alongside Kotlin/Wasm	
05	NODE.JS / SERVERLESS	Fast cold start · kotlinx-nodejs typed APIs	

Not Kotlin React. Not Kotlin Node. A typed contract for any web team.

// THE REST OF THIS TALK IS WHAT TO DO AT THE BOUNDARY

// NOT FLUTTER. NOT REACT NATIVE.

WE DON'T SHARE THE UI.

// SHARED — THE LOGIC

networking · repositories · caching
API models · auth · feature flags
offline sync · analytics · pagination
validation rules

→ written once. Not twice.

// NATIVE, ALWAYS

SwiftUI stays SwiftUI.
Compose stays Compose.
Native gestures, scroll, a11y.
The phone still feels like the phone.

→ iOS engineers stop treating it as foreign.

// WHY NOW

Old rough edges fixed — new memory manager (no more frozen shared objects), clean Swift interop (suspend fun → try await).
In production at Cash App, Netflix, McDonald's.

// SHARE WHAT MAKES SENSE, KEEP WHAT MATTERS NATIVE

// API vs SDK

SHIP AN SDK, NOT AN API.

// API → A CONTRACT

Just URLs you can call.

Every client re-implements:

fetch · parse · error states.

→ they drift, platform by platform.

// SDK → A LIBRARY

TypeScript types — free (.d.mts).

Zero fetch / parse boilerplate.

You don't learn Kotlin.

→ it's an npm install.

// THE COST — BE HONEST

≈ 800 KB bundle. Dead weight on a static landing page — pays off on a real app already shipping Android + iOS.

// FRONTEND DEV: IT'S JUST AN npm PACKAGE WITH GREAT TYPES

ACT

01 THE DREAM

ONE SOURCE OF TRUTH FOR EVERY PLATFORM.

// 05:30

// THE COMPILE PIPELINE

HOW .KT BECOMES .JS.

01
.kt
SOURCE
commonMain + jsMain

▶

02
IR
COMPILER
js(IR) backend

▶

03
.js
OUTPUT
.mjs + .d.mts

▶

04
V8
RUNTIME
browser / Node

// ONE expect, FOUR actuals — THE SEAM, NEXT SLIDE

// THE PLATFORM SEAM

ONE expect, FOUR actuals.

```
expect fun createHttpClient(): HttpClient
```

one actual per target

ANDROID	iOS	JVM	JS
OkHttp	Darwin	CIO	Ktor JS

// KMP PICKS THE ENGINE; YOUR CODE NEVER KNOWS

// WHAT DOES js(IR) ACTUALLY DO?

FOUR LINES OPEN THE BROWSER.

01	`browser()`	runs in any browser — and Node, with a webpack rewrite.
02	`useEsModules()`	modern ESM imports — tree-shakeable.
03	`generateTypeScriptDefinitions()`	auto-emitted .d.mts — TS users get types for free.
04	`binaries.executable()`	a runnable bundle webpack picks up.

// KOTLIN 2.2 NEEDED -Xes-long-as-bigint — 2.3 MADE Long→BigInt THE DEFAULT

// JUST @JsExport EVERYTHING?

EXPORT THE BEHAVIOR.

HIDE THE MACHINERY.

// IN THIS REPO

13

@JsExport.Ignore annotations

Each one a small "you don't need to see this from JS":

- StateFlow
- CoroutineScope
- Repository constructors

Exported: load · subscribe · onCleared

// subscribe(callback) IS THE BRIDGE — DETAILS IN ACT 2

// KOTLIN RUNNING IN V8

KOTLIN CODE, NODE ENGINE.

```
$ node server.js
✅ KMP Library initialized successfully.
🚀 Demo API server listening on http://localhost:3000

$ curl localhost:3000/modules/lines
{
  "lines": [
    { "designation": "10", "name": "Blå linjen", "transportMode": "METRO" },
    { "designation": "13", "name": "Röda linjen", "transportMode": "METRO" },
    { "designation": "17", "name": "Gröna linjen", "transportMode": "METRO" },  …

// 67 lines of business glue around the same LinesViewModel the app uses.
```

// SAME REPOSITORY. SAME DTO. SAME ERROR MAPPING. ENGINE: KTOR JS / NODE

ACT

02 THE REALITY

WHERE COROUTINES MEET REACT.

// 12:30

// THE MATRIX SHIFTS · THE RULE SURVIVES

THE MATRIX SHIFTS. CHECK klibs.io.

// ROOM

Annotation-first.
Google's library.

Targets: JVM · Native · JS · Wasm

// JS + Wasm — RECENTLY ADDED

// SQLDELIGHT

SQL-first.
Pluggable driver, every platform.

Targets: JVM · Native · JS · Wasm

// Web Worker runs SQLite-in-Wasm

// THE RULE

Check klibs.io for the JS / Wasm badge BEFORE you commit a dependency. iOS hides under the Kotlin/Native badge — don't read it as 'unsupported'. The ecosystem moves quarterly; the discipline doesn't.

// EARLIER, THIS SLIDE SAID 'ROOM CAN'T.' IT AGED IN OUR FAVOR — GOOGLE SHIPPED JS + WASM FOR ROOM.

// StateFlow IS NOT JS-FRIENDLY

StateFlow → A CALLBACK.

```
fun subscribe(onStateUpdate: (T) → Unit) {
    viewModelScope.launch {
        uiState.collect { onStateUpdate(it) }
    }
}
```

One method. Framework-agnostic. The whole bridge from Kotlin's side.

// WON'T KOTLIN ASYNC FIGHT WITH REACT?

Dispatchers.Main IS THE MICROTASK QUEUE.

`emit()` → `queueMicrotask() in V8` → `your setState`

// = Promise.resolve().then(setState) — SAME QUEUE, NO COMPETITION

// WHY REACT, NOT COMPOSE WEB?

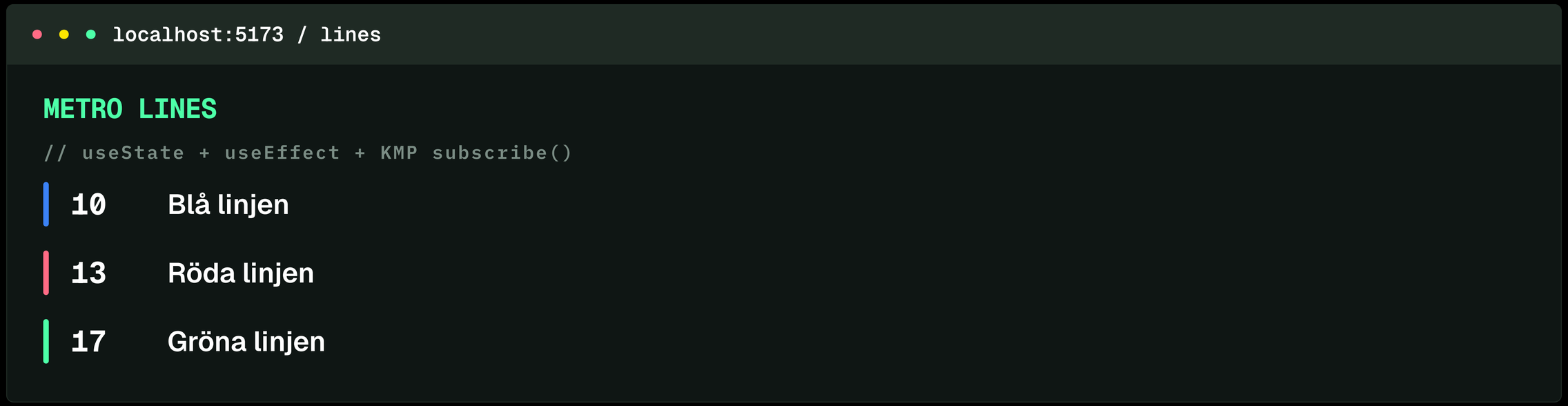
12 LINES OF TYPESCRIPT.

```
function useStockholmTransport(factory, load) {
    const [state, setState] = useState(null)
    useEffect(() => {
        const vm = factory()
        vm.subscribe(setState)   // ← the bridge
        load(vm)
        return () => vm.onCleared()   // ← the contract
    }, [])
    return state
}
```

// PROOF THE SDK DOESN'T LEAK KOTLIN — IN REACT, subscribe IS A PLAIN JS CALLBACK

// THE SAME KMP MODULE, IN A BROWSER TAB

SAME SDK, BROWSER UI.



// SAME VIEWMODEL THE PHONES SUBSCRIBE TO. ZERO BUSINESS LOGIC IN THE BROWSER.

// FORGET onCleared(), WATCH IT LEAK

ONE LINE BETWEEN A LIBRARY AND A LEAK.

// WITHOUT cleanup

```
useEffect(() ⇒ {
  vm.subscribe(setState)
})
```

1 → 2 → 4 → 12

subscribers, climbing every nav.

// WITH cleanup

```
useEffect(() ⇒ {
  vm.subscribe(setState)
  return () ⇒ vm.onCleared()
})
```

1 → 1 → 1

flat. coroutine scope = contract.

ACT

03 THE PAYOFF

SHIPPING IT.

// THE PACKAGE.JSON IS THERE — JUST UNSTYLED

ALMOST.
HERE'S THE POLISH.

// THE GAPS

- **Scope — still mirrors the Gradle dir name**
- **Missing `module` + `exports` fields**
- **Empty `peerDependencies`**
- **`main` + `types` ARE there. Newer Kotlin = more.**

All closes through the packageJson { } DSL — half a day of polish, not research.

// WHAT REAL LOOKS LIKE

@jacksonmafra-umain/stockholm-transport

ESM (browser) + commonjs2 (Node)

peerDeps: Ktor JS · Koin · coroutines

One build, four registries:

Maven · SPM · npm (npm-publish plugin)

// JETBRAINS' OFFICIAL GUIDE — LINK ON THE LANDING PAGE

// FIREBASE · STRIPE · SENTRY — THE TOKEN PATTERN

WHO'S USING YOUR SDK?

`init(apiKey)` → SDK stores it → injects header → **backend validates + attributes**

Same pattern: Firebase (google-services.json) · Stripe (pk_ / sk_) · Sentry (DSN)

⚠ **TOKEN ≠ USER AUTH**

Identifies the consuming app, not the human.

Never ship a raw API key in a JS bundle — anyone reads it in DevTools. Backend proxy, or runtime-inject via DI (Koin, in this repo).

// FOR PER-USER IDENTITY, LAYER OAUTH ON TOP

// FIX. RELOAD. DROP MIC.

FIX ONE LINE.
THREE PLATFORMS.

1.	BREAK	LinesRepositoryImpl.kt — wrong query param. Save.
2.	PUBLISH	:stockholm-transport:publishToMavenLocal + jsBrowserDistribution
3.	FAIL	Android · iOS · browser — all three error (boop!)
4.	FIX	Undo the line. Re-publish.
5.	RELOAD	All three reload. Lines come back.

// THREE RULES TO TAKE HOME

THREE RULES.

01 Export the BEHAVIOR. Hide the MACHINERY.

02 Coroutine SCOPE = lifecycle CONTRACT.

03 The SDK boundary is the SAME on every platform.

Why build it twice? Share what makes sense, keep what matters native.
The web is just the next front door.

// TACK · DĚKUJI · THANK YOU

THANK YOU.
TACK.
Děkuji.

// THE REPO — CODE · SLIDES · DEMOS

github.com/jacksonmafra-umain/StockholmTransport

// SCAN FOR THE REPO

// FIND ME

Jackson Mafra · Umain
jackson.mafra@umain.com